

George Bibu JSONPlaceholder API Test

GitHub Link: [ust-sdet-training/SDET at george_25jun26](https://github.com/ust-sdet-training/SDET-at-george_25jun26)

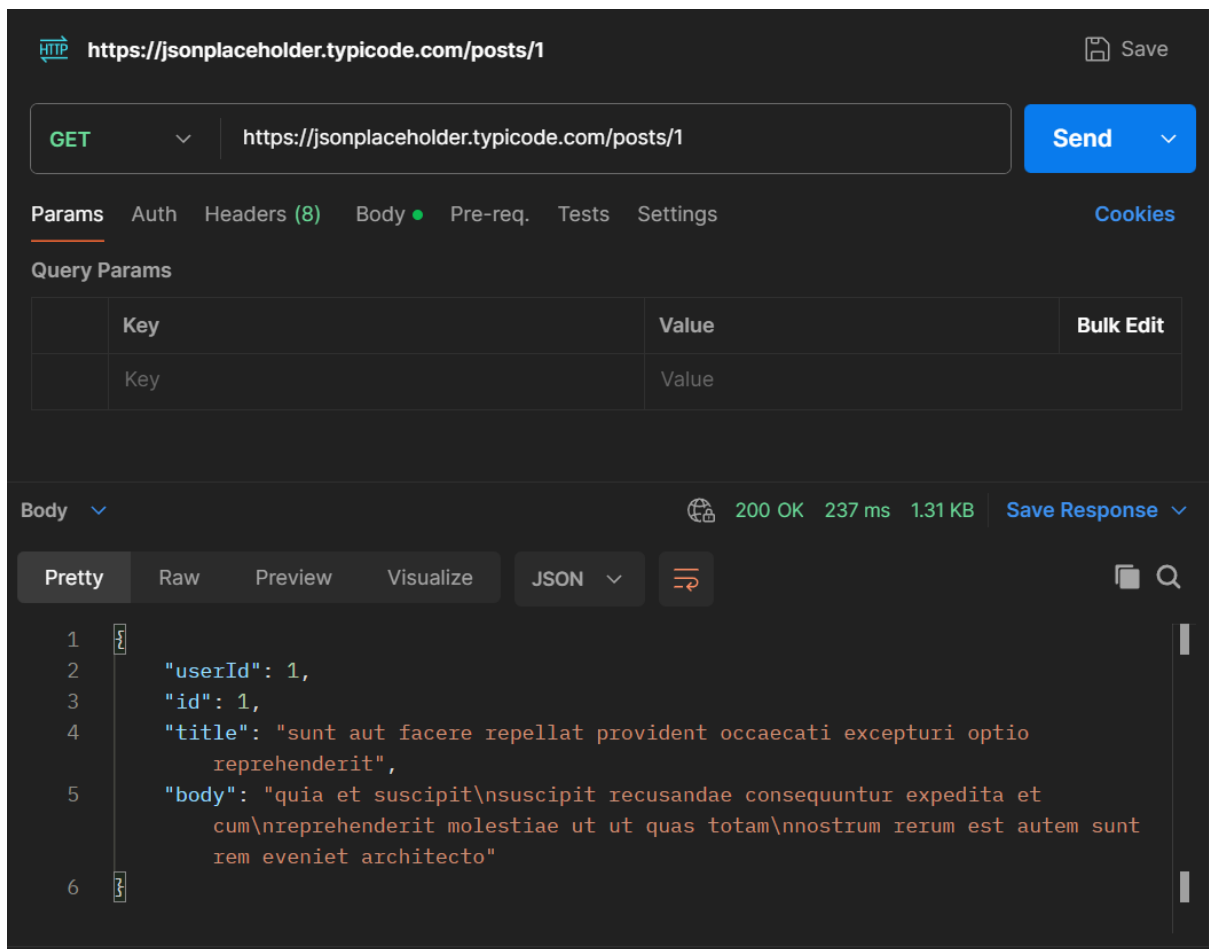
Objective: To test GET, POST, PUT and DELETE API endpoints for the website

<https://jsonplaceholder.typicode.com>

Test Case Flow: A single file called JSONPlaceholderTest.java is created to run four different tests for GET, POST, PUT and DELETE requests for the website.

1) GET request

The first test is for a GET request. A get request is created to fetch a resource from the site.



GET request in Postman

A path parameter for 'id' is set to 1 and the body of the received response is asserted to contain the exact same 'userId', 'id', 'title', and 'body' as seen in the Postman response. Status code is asserted to be 200.

```

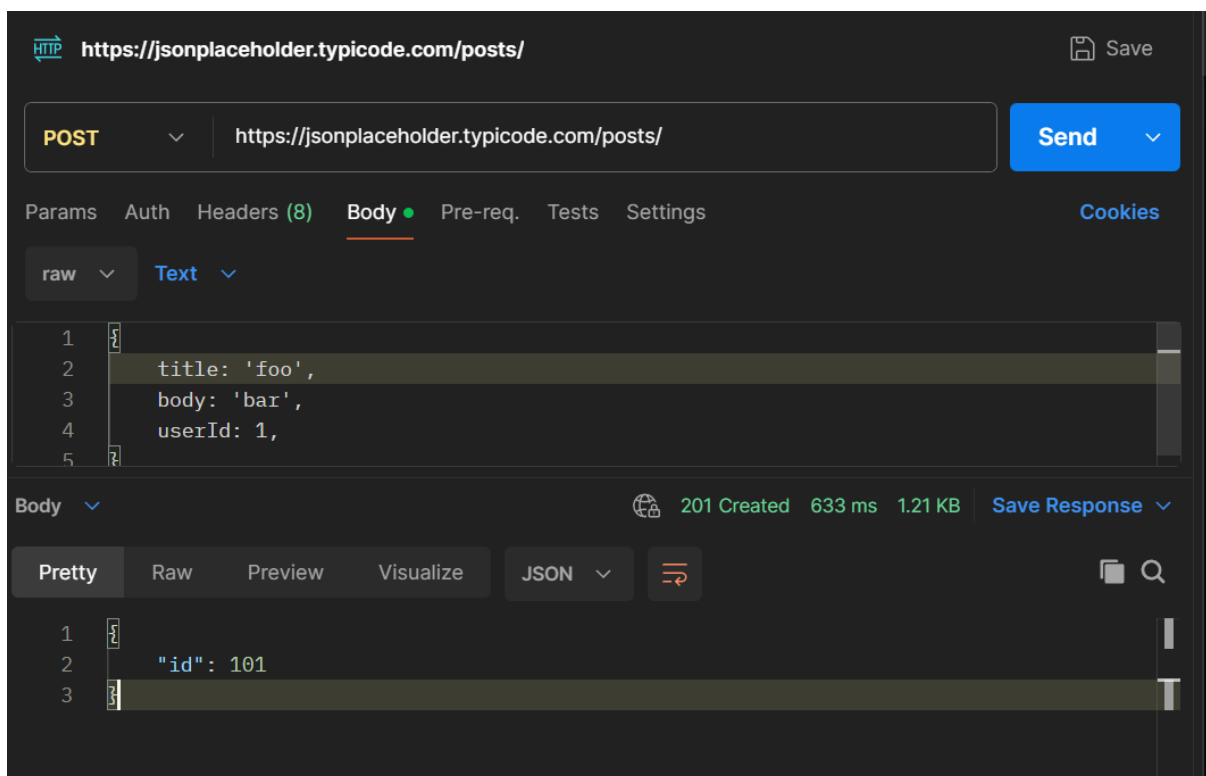
public class JSONPlaceholderTest {
    @Test
    @DisplayName("Get body from JSONPlaceholder site with id 1 using GET")
    void getResponse() {
        given()
            .pathParam("id", 1)
        .when()
            .get("https://jsonplaceholder.typicode.com/posts/{id}")
        .then()
            .statusCode(200)
            .body("userId", equalTo(1))
            .body("id", equalTo(1))
            .body("title", equalTo("sunt aut facere repellat provident occaecati excepturi optio reprehenderit"))
            .body("body", equalTo("quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas tota\nperferendis dolor a ea maiores delectat autem ut repellat repellendus neque molestias omnis voluptas maiores sunt blanditiis dolorum non est a ea"))
    }
}

```

Code snippet for the GET request test

2) **POST request**

The second test is for a POST request. With this POST request we are creating a resource using details such as 'title', 'body' and 'userId'. The body of the request contains the details of the resource we wish to add using Map.of().



POST request in Postman

The response body of a successful POST request contains the 'id' of the resource created. The response is asserted to contain this id and status code is asserted to be 201.

```

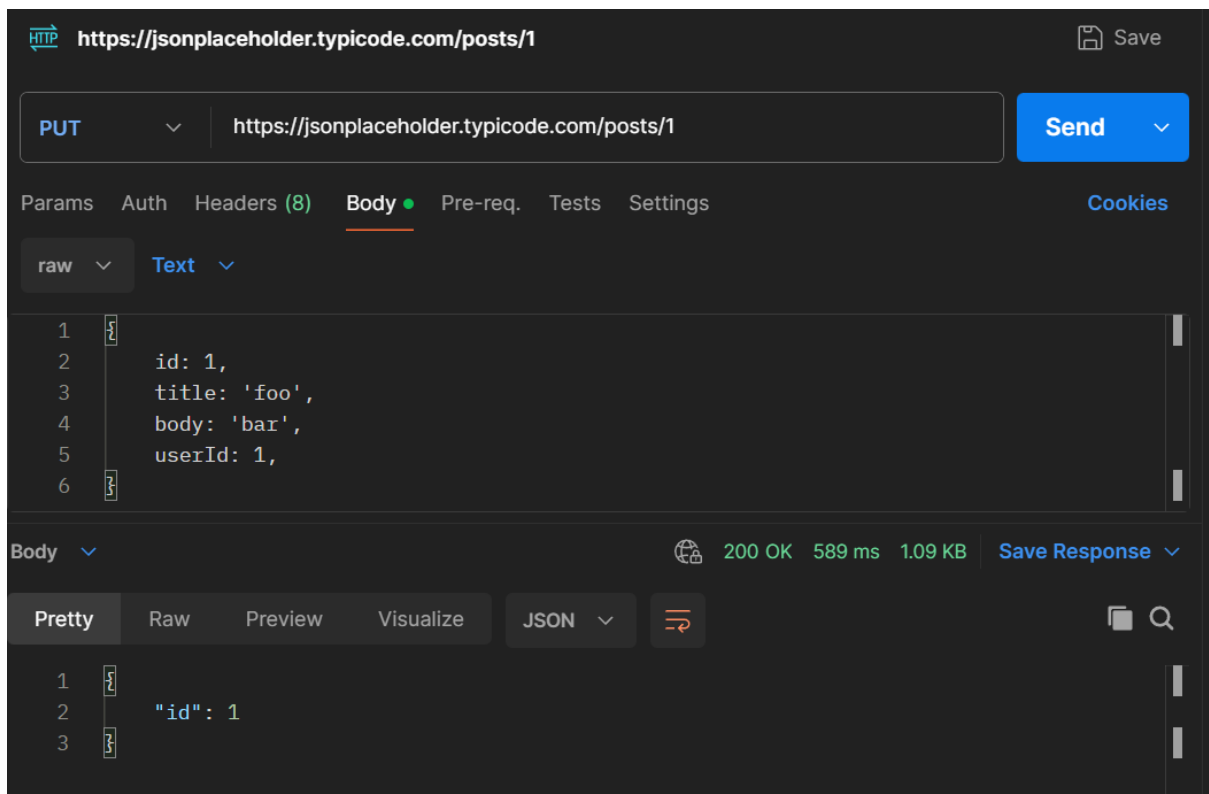
@Test
@DisplayName("Create a resource using POST")
void postResponse() {
    given()
        .body(Map.of("title", "foo",
                     "body", "bar",
                     "userId", 1
                    ))
        .when()
            .post("https://jsonplaceholder.typicode.com/posts/")
        .then()
            .statusCode(201)
            .body("id", equalTo(101));
}

```

Code snippet for the POST request test

3) PUT request

The third test is for a PUT request. A resource's fields are updated using this request. The body of the request contains the updated details of the resource we wish to update using Map.of(). For the PUT request, we set an 'id' path parameter to specify which resource we want to update.



PUT request in Postman

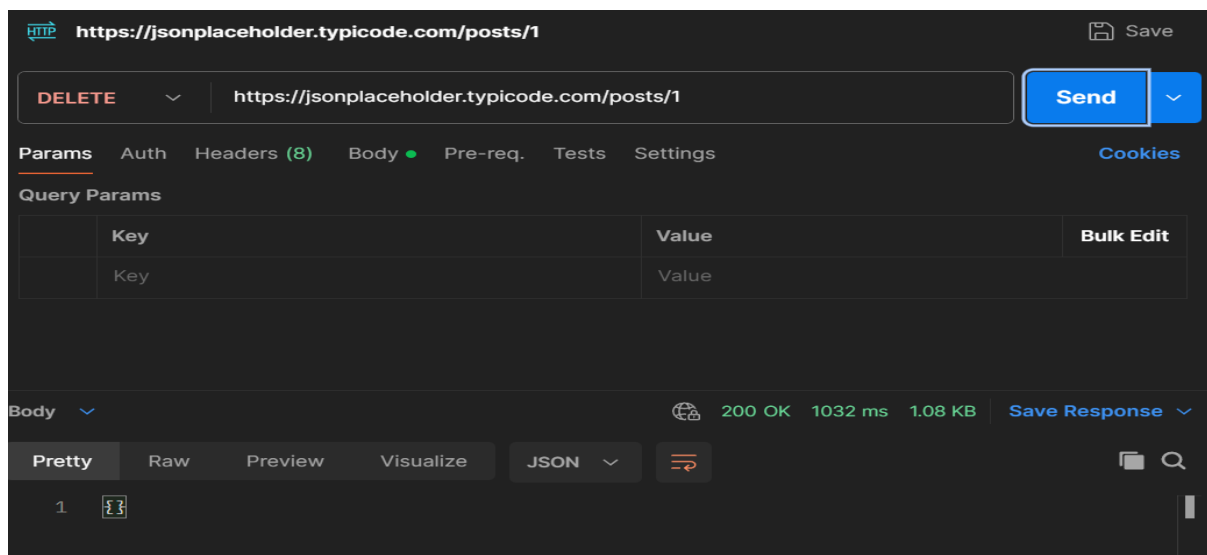
A successful PUT request returns the id of the updated resource. The response is asserted to contain this id and the status code is asserted to be 200

```
@Test
@DisplayName("Update resource using PUT")
void putResponse() {
    given()
        .body(Map.of("id", 1,
            "title", "foo",
            "body", "bar",
            "userId", 1
        ))
        .pathParam("id", 1)
    .when()
        .put("https://jsonplaceholder.typicode.com/posts/{id}")
    .then()
        .statusCode(200)
        .body("id", equalTo(1));
}
```

Code snippet of PUT request

4) DELETE request

The final test is for the DELETE request. For a DELETE request, we only need to specify the resource we wish to delete using its 'id' in the path parameter.



Delete request in Postman

A successful DELETE request only returns a status code of 200. The response is asserted to have this status code.

```
@Test
@DisplayName("Delete a resource using DELETE")
void deleteResponse() {
    given()
        .pathParam("id", 1)
        .when()
        .delete("https://jsonplaceholder.typicode.com/posts/{id}")
        .then()
        .statusCode(200);
}
```

Commands:

- 1) Run tests: `mvn clean test -Dtest=JSONPlaceholderTest`

Test Results:

```
[INFO] -----
[INFO]  T E S T S
[INFO] -----
[INFO] Running JSONPlaceholderTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.698 s -- in JSONPlaceholderTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 9.504 s
[INFO] Finished at: 2026-06-25T12:14:34+05:30
[INFO] -----

C:\Users\310918\Test\SDET\APITesting>mvn clean test -Dtest=JSONPlaceholderTest
```